O&G Workbench v0.9.4 — Independent read-only audit

4 November 2026 · Release stabilization · No application changes or PR

Assessment

Do not release this archive unchanged. The principal blockers are destructive import boundaries: fault Merge can change topology, 297 imports can mix depth units or remove prior children, and an unusable horizon placement can clear an existing horizon under Replace. A separate stale catalog-size assertion makes the real Geo self-test fail and would block the configured Windows build gate. These call for targeted fixes and regression tests, not a redesign or a renderer/framework rewrite.

The audit records **15 findings: 4 Critical, 5 High, 5 Medium, 1 Low**. Each is supported by current source and a controlled reproducer; severity reflects the stated trigger and potential impact, not evidence that a client has already experienced it.

Baseline, source identity and limits

The authoritative artifact is the uploaded OandG_Workbench_v0.9.4.zip, whose application version is 0.9.4. SHA-256: 883a01dd8486e9166fd32dba9b38edf329a18da550ac54c12aa85d558039ffeb.

The brief names branch claude/new-session-96rnbt. The archive has no Git commit identity; equivalence to a particular branch HEAD was not established. Results apply to this exact ZIP, not an independently fetched branch revision. No v0.9.3 source checkout was supplied, so “introduced in v0.9.4” is asserted only where the changed catalog contract itself establishes a regression. Existing 297 defects are reported as current defects, without inventing their introduction date.

All **613 original files remain byte-identical**, with no missing or extra files in the extracted source tree. The archive contains 435 Python files and 199,214 Python lines including tests. Inspection targeted the entire supplied audit brief, shared boundaries and the native-import/symbol workflows; the entire available automated suite was run. This is not an assertion of exhaustive line-by-line verification or manual testing of every screen. Synthetic projects, probes, settings, dependencies, logs and reports were outside the application tree. No code, schema, scientific formulas or supplied client project was modified; no PR was created.

Tests ran on Linux/Python 3.12 with a headless Xorg Tk display. The external environment included NumPy 2.3.5, Matplotlib 3.10.8, pyproj 3.8.0, segyio 1.9.15, pyshp 3.1.6, lasio 0.32, Pillow, openpyxl, pypdfium2 and pytest 9.2.0. Windows behavior was reviewed in source/build configuration; no Windows or frozen executable was executed.

Full-suite result

1,827 passed, 1 failed, 9 skipped; 521 subtests passed; 825.01 seconds. The reported 1,835-pass baseline was not reproduced exactly. This does not mean eight product regressions were found: test counts depend on the fixtures/platform and the exact branch revision, which was not independently identified here. JUnit totals include subtests and should not be added to pytest’s test count.

The sole failure is tests/test_text_style.py::ControlTests::test_the_common_control_and_tk_fonts, expecting a bold Tk font but getting normal. It reproduces individually. The cloud Tk runtime does not have the requested DejaVu Serif family and substitutes fixed/normal, although Matplotlib lists its own font. Treat this as an environment-dependent assertion, not a demonstrated Windows product-font defect. The failure is retained in the logs, not silently removed from the baseline.

The nine skips are seven unavailable real-raster/autoreg fixtures, the Windows Explorer drag/drop test and the opt-in packaged PDF build. Their exact names/reasons are in skipped-tests.json. **No raster-process crash occurred in this full run.** One passing run does not explain the earlier reported crash or establish that it cannot recur.

The native Kingdom, 297/298 and symbol test families ran successfully in this baseline. Additional probes below expose cases those tests do not cover. Separately, the real source app.py --self-test exits **1** on the stale >=100-symbol assertion (H01); the mostly passing pytest suite does not override that result.

Findings index

ID	Severity	Finding
C01	Critical	Fault Merge collapses separate sticks and interleaves their ordered nodes
C02	Critical	IHS 297 writes feet-valued tops into metre wells while converting the survey
C03	Critical	A header-only 297 fill import deletes earlier tops, tests and cores
C04	Critical	Replace with an out-of-survey horizon origin clears and saves the existing horizon
H01	High	The approved 29-symbol catalog makes the build self-test fail
H02	High	Rejected duplicate-code edits still change and persist status mappings
H03	High	Valid JSON with malformed catalog fields can prevent Geo and Seismic startup
H04	High	Each newly created 297/298 well triggers another full well-table scan
H05	High	Native Kingdom imports overwrite a persistence-failure warning with success
M01	Medium	Corrupt native files can escape the promised import diagnostic path
M02	Medium	Native horizon origin placement silently assumes unit line/trace increments
M03	Medium	Duplicate SYM numbers save successfully and make numeric resolution order-dependent
M04	Medium	Existing Map artists retain an old catalog style after catalog installation
M05	Medium	A picked native HZN cell with a non-finite value is counted and imported without validation
L01	Low	Save closes the Symbol Editor without warning about unapplied form edits

Critical

C01 — Fault Merge collapses separate sticks and interleaves their ordered nodes

Severity: Critical

Module / files and lines: seismicworkbench/kingdom.py:673–710,816–836; seismicworkbench/interpretation.py:118–134

Function / class: fault_picks / apply_faults / fault_sticks

Exact problem: Stick identity in the renderer is the section string, derived from a section label plus native stick_id. Merge concatenates incoming picks without giving their sticks a collision-free identity. Reused stick IDs/section labels from another file or another import therefore group together; sorting by seq interleaves them. Merge also replaces, rather than accumulates, the fault provenance.

Why it matters: This violates Fault → Stick → ordered picks and can persist a geometrically different fault. Retained old nodes can lose their original source attribution.

Reproducible scenario: Import one three-node stick with z=[100,101,102], then Merge a different source whose stick has the same native ID and section with z=[200,201,202]. After actual database close/reopen, one stick remains, ordered [100,200,101,201,102,202]; provenance lists only source1.FLT.

Recommended correction: Use a stable source/import-scoped stick key separate from its display label. Under Merge preserve independent stick groups, define intentional duplicate/reimport behavior, and retain provenance for all contributing sources.

Regression test: Merge two non-identical same-ID sticks and reimport the same file; compare ordered groups and source identities before/after close/reopen for New/Merge/Replace.

Should existing tests have caught it?: test_kingdom_native.py checks single-import topology and reopen; its app-flow existing-name test exercises New name, not colliding Merge. Those tests cannot establish this invariant.

Evidence: fault-persistence-evidence.json: merge_reopen; boundary-evidence.json: fault_merge

Confidence: High

C02 — IHS 297 writes feet-valued tops into metre wells while converting the survey

Severity: Critical

Module / files and lines: ogcore/ihs297.py:384–413,485–497,504–594,619–643,758–914

Function / class: header_values / tops / ip_test / fm_test / cores / apply / _children

Exact problem: The source length convention is feet. store_survey is given length_unit="ft" and converts correctly, but tops, test intervals and core intervals are written as raw numbers. Existing-well fill keeps the well's metre unit. Header depth/elevation handling can also mix source and target conventions.

Why it matters: Related records within one well no longer share a physical depth system. This can misplace tops/zones/tests relative to the trajectory and logs. In overwrite mode, assigning elevation_units="ft" is also not a conversion of existing metre data.

Reproducible scenario: On an existing metre well, import a 1,000-ft top and a vertical survey ending at 1,000 ft. The survey is stored at 304.8 m; the top is stored at 1,000 m.

Recommended correction: Convert every incoming length field to the established target well unit before writing, including elevations, TD, tops, cores and tests, or reject a unit mismatch explicitly. Do not change the well unit without a complete, explicit data conversion.

Regression test: Feet and metre existing wells in fill/overwrite modes; compare all related MD/TVD/elevation/recovery fields to hand conversions after reopen. Include wells with existing interpreted metre data.

Should existing tests have caught it?: test_native_ihs297.py validates feet-based fixture wells and survey handling but has no metre-target integration case. Its counts/idempotence checks would still pass.

Evidence: boundary-evidence.json: 297_metric_well

Confidence: High

C03 — A header-only 297 fill import deletes earlier tops, tests and cores

Severity: Critical

Module / files and lines: ogcore/ihs297.py:803–809,835–847,875–914

Function / class: apply / _children

Exact problem: For every included existing well, _children unconditionally deletes IHS 297 tops, tests and cores before writing incoming children. The deletion is source-wide, independent of the selected file and whether that record family appears in it. The comment says this file's earlier import is replaced, but the predicate is broader.

Why it matters: A partial delivery can erase prior project records during a fill import, with no child-deletion preview. Prior tests have source_file provenance, but deletion does not use it; tops and cores have only broad vendor markers.

Reproducible scenario: Import a well with an IHS top. Import a second parsed file containing the same well header and no tops using default fill policy. The existing top count goes from one to zero. The same unconditional deletion applies to IHS tests and marked cores.

Recommended correction: Preserve absent record families in fill/partial-update workflows. Scope replacement to explicit source/file or stable record identities, and require an explicit complete-snapshot replacement policy with a deletion preview when absence means deletion.

Regression test: A header-only and a tests-only second delivery must preserve unrelated prior children; explicitly authorized full replacement should delete only the intended records and roll back on failure.

Should existing tests have caught it?: test_reimport_is_idempotent_and_no_duplicate_wells compares counts for the same complete file. It does not test partial deliveries or different file provenance; the existing fill test preserves only another interpreter's tops.

Evidence: boundary-evidence.json: 297_header_only_reimport

Confidence: High

C04 — Replace with an out-of-survey horizon origin clears and saves the existing horizon

Severity: Critical

Module / files and lines: seismicworkbench/app.py:1212–1288; seismicworkbench/kingdom.py:787–813

Function / class: import_kingdom_horizons / apply_horizons

Exact problem: The selected origin is parsed but a resulting empty placement is still submitted under Replace. apply_horizons clears the old grid before assigning zero incoming picks. The app then saves and reports a successful zero-pick import.

Why it matters: A wrong origin or incompatible survey can erase a valid interpreted surface instead of rejecting an import that cannot place anything. Preview placement uses the offered origin before the user's final edited origin.

Reproducible scenario: Through the actual app import method with its preview callback: first import H02503 at 6448/3460 (1,830 picks), then choose 0/0 and Replace on the same survey. The horizon becomes empty, save is called, and the status says 10300 (0 picks).

Recommended correction: Recompute and validate placement after the final origin choice and before mutation. Reject zero-usable-pick replacement and preserve the old horizon; an intentional clear must be a separate explicit action. Report partial coverage before destructive replacement.

Regression test: Wrong origin, all picks outside and a valid partial overlap under every policy; assert old grids and persisted data survive rejected imports.

Should existing tests have caught it?: Current native app tests cover missing units, different sizes, New name and Cancel. They do not combine an edited invalid origin with Replace.

Evidence: horizon-app-evidence.json; probes/horizon_app.py

Confidence: High

High

H01 — The approved 29-symbol catalog makes the build self-test fail

Severity: High

Module / files and lines: geoworkbench/main_window.py:2318–2326; Build_Windows_EXE.bat:44–57

Function / class: self_test / Windows build smoke-test gate

Exact problem: Geo's self_test asserts len(SYMBOLS)>=100 although the shipped active catalog has exactly the required 29 symbols. The Windows build invokes that self-test and aborts on its nonzero exit.

Why it matters: The release build gate rejects a valid shipped catalog. This is a concrete stale assertion, not evidence that the WSD is missing or that Windows itself was executed here.

Reproducible scenario: Run app.py --self-test with the unmodified archive and a fresh user settings path. It exits 1; the application log records AssertionError: well symbol catalogue (.WSD) missing at the >=100 assertion.

Recommended correction: Validate the required shipped resource/catalog contract rather than the obsolete size threshold; distinguish resource presence from user-added/deactivated entries. Retain the failure gate.

Regression test: Execute the real source self-test and frozen Windows self-test with the 29 shipped codes, fresh settings and valid edited catalogs.

Should existing tests have caught it?: test_exactly_the_29_approved_symbols passes, while PDF packaging tests mostly inspect build configuration. No default suite test executes this real Geo self-test end-to-end.

Evidence: geo-self-test-application.log; geo-self-test.log; CLI exit code 1

Confidence: High

H02 — Rejected duplicate-code edits still change and persist status mappings

Severity: High

Module / files and lines: geoworkbench/symbol_catalog_dialog.py:264–299

Function / class: SymbolCatalogDialog.apply_form

Exact problem: apply_form rewrites mappings and shared-geometry references before validation. On rejection it restores only the edited entry. For GAS→OIL, the duplicate rename is rejected, but IHS GAS mappings now point to OIL and the restored catalog validates successfully.

Why it matters: A user can save a change that the editor explicitly refused. Subsequent IHS imports can assign oil symbols to gas wells; a malformed-name attempt can instead leave the working catalog unsavable.

Reproducible scenario: Load GAS, change Code to OIL, click Apply and receive the duplicate-code error. GAS is restored, but the IHS GAS mapping is OIL. Save, reload the catalog: the wrong mapping persists.

Recommended correction: Validate a complete candidate copy before publishing any changes, or roll back every dependent mapping/geometry reference together with the entry. Rejection must leave the entire working catalog unchanged.

Regression test: Snapshot all entries/mappings/render references before failed duplicate/invalid/cyclic renames and assert complete equality afterward, then Save/restart and resolve representative statuses.

Should existing tests have caught it?: test_duplicate_code_and_invalid_alias_are_refused asserts only that the GAS entry is restored, not that dependent mappings remain unchanged.

Evidence: rename-evidence.json; editor-evidence.json: rejected_rename

Confidence: High

H03 — Valid JSON with malformed catalog fields can prevent Geo and Seismic startup

Severity: High

Module / files and lines: ogcore/symbol_catalog.py:192–205,216–258,320–325; geoworkbench/main_window.py:136; seismicworkbench/app.py:62

Function / class: symbol_catalog.load / current / application constructors

Exact problem: load catches file/JSON parsing errors, then assumes symbols is a mapping and directly converts symbol_id/scale/show counts. A structurally valid JSON file with scale="oops" or symbols as a list raises uncaught ValueError/AttributeError. Invalid render definitions can also be installed without validate().

Why it matters: A bad local settings file becomes an application startup dependency. There is no safe fallback/diagnostic at the field-validation boundary for these cases.

Reproducible scenario: Load {"symbols":{"OIL":{"scale":"oops"}}}: ValueError. Load {"symbols":["x"]}: AttributeError. These reach current(), called during Geo and Seismic initialization. An unknown body loads/installs even though validate subsequently identifies it.

Recommended correction: Validate schema/types and the complete catalog before installation. Preserve the malformed file for diagnosis, surface a useful settings error and use a known-safe catalog rather than crashing or installing invalid geometry.

Regression test: Malformed but parseable JSON for every field/container, unknown geometry, bad mappings, NaN/Inf scales and future versions; open Geo/Seismic with fresh and malformed settings in source/frozen layouts.

Should existing tests have caught it?: Catalog tests cover valid persistence/import and editor validation. They do not exercise malformed on-disk user settings through startup.

Evidence: boundary-evidence.json: bad_numeric, bad_symbols_type, bad_geometry

Confidence: High

H04 — Each newly created 297/298 well triggers another full well-table scan

Severity: High

Module / files and lines: database.py:1668–1708; ogcore/ihs298.py:529–549; ogcore/ihs297.py:786–794

Function / class: refresh_api14 / api_owners / add_well / native import apply loops

Exact problem: Each add_well duplicate check calls refresh_api14, whose UPDATE predicate compares api14_src with api_uwi across the table. EXPLAIN reports SCAN wells even when every normalized API is already current.

Why it matters: A batch creating M wells in an N-well project performs roughly N×M normalization checks in addition to inserts. IHS 298's creation loop also precedes its monthly-row progress/cancellation callbacks, so a large new-well phase can remain unresponsive.

Reproducible scenario: Creating 100 wells through IHS 298 produced 100 full-table refreshes and one commit. Apply took 0.0225 s on an empty project and 0.8063 s with 112,000 seeded wells. Planning took 0.0084/0.1151 s. These are controlled observations, not a full-file throughput guarantee.

Recommended correction: Refresh/validate normalization once at a safe bulk boundary and maintain known-clean inserted/updated rows, while keeping indexed exact-API duplicate checks. Add progress/cancellation during well creation. Do not remove identity protection.

Regression test: Trace a bulk import and bound full-table refresh count independently of imported well count; include prior raw SQL API edits, duplicate protection, 100k-well projects and cancellation during creation.

Should existing tests have caught it?: Existing tests establish an index on exact API lookup and import correctness, but do not catch the separate full-table refresh before every indexed lookup.

Evidence: performance-evidence.json; probes/performance.py

Confidence: High

H05 — Native Kingdom imports overwrite a persistence-failure warning with success

Severity: High

Module / files and lines: seismicworkbench/app.py:1286–1289,1355–1363,1379–1393

Function / class: import_kingdom_horizons / import_kingdom_faults / save_now

Exact problem: save_now catches sqlite3.Error and writes Interpretation not saved yet, without returning a failure outcome. Native import methods immediately replace that status with Kingdom ... imported and return success.

Why it matters: An import can be present only in memory while the user is told it succeeded. The old database transaction may remain safe, but durability failure is hidden and closing can lose work.

Reproducible scenario: Inject sqlite3.OperationalError("synthetic disk full") at save_interpretation while using the actual app save_now/import methods. Status first reports the error, then says Kingdom horizons imported: 10300 (1,830 picks); an ordinary success result is returned.

Recommended correction: Return/propagate a durable-save outcome and preserve a prominent unsaved/error state. Only report a saved import after successful persistence; define retry/rollback behavior without discarding the working interpretation.

Regression test: Inject save failures for horizons and faults through the app path, assert old durable data survives, the error remains visible, no false durable success is returned, and a successful retry clears the unsaved state.

Should existing tests have caught it?: Native app-flow tests replace save_now with a counter. Store round-trip tests validate successful writes; neither catches this app/store failure interaction.

Evidence: save-failure-evidence.json; injected I/O/database failure, not observed client disk failure

Confidence: High

Medium

M01 — Corrupt native files can escape the promised import diagnostic path

Severity: Medium

Module / files and lines: seismicworkbench/kingdom_native.py:105–123,222–243; seismicworkbench/kingdom.py:476–588; ogcore/cfb.py:114–155

Function / class: read_fault / _directories / read_native

Exact problem: Directory entries are unpacked without checking available bytes. read_fault wraps CFBError only from opening the container, not later stream reads. read_native catches KingdomNativeError only.

Why it matters: A truncated horizon or damaged FLT stream can raise struct.error/CFBError directly instead of producing an unreadable-file diagnostic. In a mixed selection it interrupts preview of otherwise valid files. No database corruption was established by this probe.

Reproducible scenario: Truncate H02503 at 1,750, 2,400 or 4,200 bytes: unpack_from raises outside read_native handling. Mutate an FLT Fault_Data directory start to an invalid sector: CFBError escapes while reading the stream.

Recommended correction: Check every declared count/range before decoding and normalize expected malformed-container/stream exceptions into the native importer error/diagnostic type. Include path and offending region; do not broadly suppress programming errors.

Regression test: Truncate around every directory/sector/tile boundary, corrupt stream references/counts, and import a mix of valid/bad files. Assert useful diagnostics and unchanged project data.

Should existing tests have caught it?: Current not-a-file tests reject invalid signatures; a few invalid outer headers do not cover inner truncation/stream failures.

Evidence: native-edge-evidence.json: truncate_*; fault-persistence-evidence.json: broken_flt

Confidence: High

M02 — Native horizon origin placement silently assumes unit line/trace increments

Severity: Medium

Module / files and lines: seismicworkbench/kingdom.py:104–127,635–667; seismicworkbench/app.py:1203–1212

Function / class: KingdomHorizon.with_origin / place_horizon / origin defaulting

Exact problem: with_origin numbers native grid indices by adding origin+index. No increment is used or checked. Equal grid shape enables an origin default even when selected survey line/trace numbers increment by 2 or are otherwise non-unit.

Why it matters: On such a selected survey, some samples are dropped and others map to different bins. The supported unit-increment layout should be enforced explicitly rather than implied by matching dimensions.

Reproducible scenario: A three-node diagonal on a 3×3 grid, selected survey lines [100,102,104] and traces [200,202,204], becomes numbered [100,101,102]. Only two samples place, at bins (0,0) and (1,1), so native grid index (2,2) moves to bin (1,1).

Recommended correction: Either reject unsupported survey numbering increments with a clear diagnostic or explicitly map native grid indices using a validated survey origin and numbering interval. Do not infer compatibility from shape alone.

Regression test: Unit, doubled, descending and irregular selected survey numbering; assert correct placement or a pre-mutation rejection. Add both app preview and engine tests.

Should existing tests have caught it?: The native fixtures and geometry helper use unit increments, so current orientation tests do not exercise this accepted-input assumption.

Evidence: native-edge-evidence.json: increment_two_survey; synthetic survey-numbering boundary, not a claim about the supplied Kingdom project

Confidence: High

M03 — Duplicate SYM numbers save successfully and make numeric resolution order-dependent

Severity: Medium

Module / files and lines: ogcore/symbol_catalog.py:119–158; geoworkbench/symbol_catalog_dialog.py:319–327; well_symbols.py:161–175

Function / class: Catalog.validate / duplicate / find_symbol

Exact problem: Validation checks duplicate codes but not SYM-number ambiguity. Duplicate copies the same SYM number. find_symbol(number) returns the first matching catalog entry, even if it represents a different status/geometry.

Why it matters: Numeric symbol inputs can silently resolve to a different code after editing or reordering the catalog. Distinct code-based resolution remains unambiguous.

Reproducible scenario: Assign OIL the GAS SYM number 2; validate returns no problems and save succeeds. find_symbol("2") then returns OIL rather than GAS.

Recommended correction: Define an explicit SYM alias policy: reject/warn on conflicting active numeric identities or require a code when a numeric lookup is ambiguous. Preserve deliberate equivalent aliases only with clear semantics.

Regression test: Duplicate distinct-geometry SYM values, reorder/restart, WSD import/export and numeric importer resolution; never silently choose the first conflicting meaning.

Should existing tests have caught it?: Editor tests reject duplicate codes, and WSD tests round-trip valid builtins. Neither tests conflicting SYM values or duplicate's numeric identity.

Evidence: boundary-evidence.json: duplicate_sym

Confidence: High

M04 — Existing Map artists retain an old catalog style after catalog installation

Severity: Medium

Module / files and lines: ogcore/symbol_catalog.py:302–315; ogmap/symbols.py:42–63; ogmap/view.py:1411–1437,1530–1541; geoworkbench/main_window.py:1701–1704

Function / class: install / marker_paths / _draw_well_symbols / refresh_wells / symbol-editor saved callback

Exact problem: Catalog installation clears the marker-path function cache, but it does not invalidate existing Map symbol artists/draw keys. The editor's saved callback refreshes Well Manager rows, not open Map symbol layers.

Why it matters: A saved style can appear updated in one view while an open Map keeps the old glyph/color. This is display inconsistency; the stored well code is unchanged.

Reproducible scenario: Draw an OIL well, install a catalog changing its color from #008000 to #123456, then process events and refresh_wells. The artist remains #008000. Calling set_wells to rebuild makes it #123456.

Recommended correction: Publish a catalog revision/install event to live views and include it in symbol-layer invalidation. Rebuild only affected symbol artists, without reloading geological data.

Regression test: Keep Map, Well Manager and relevant sections open while saving catalog color/geometry/scale edits; verify artist changes and stable data-read counts, including a large-map culling cache.

Should existing tests have caught it?: Current tests check cleared marker definitions and rendering in fresh figures, rather than an already painted live Map after editor save.

Evidence: map-style-evidence.json; probes/map_style.py

Confidence: High

M05 — A picked native HZN cell with a non-finite value is counted and imported without validation

Severity: Medium

Module / files and lines: seismicworkbench/kingdom_native.py:315–343; seismicworkbench/kingdom.py:635–667,802–809

Function / class: read_horizon / place_horizon / apply_horizons

Exact problem: The mask codes correctly govern picked/unpicked status, but accepted codes are not combined with a finite-value check. A corrupted NaN/Inf pick reaches placement and assignment and counts as an imported pick.

Why it matters: A non-finite source value can clear an existing finite cell during Merge or Replace while the success count still includes it. This is separate from the three genuinely unpicked H02503 cells, which are handled correctly.

Reproducible scenario: Change one picked H02503 float to NaN while retaining its picked mask code. Native read returns 1,830 picks including one non-finite z, with no malformed-value diagnostic.

Recommended correction: Validate finite numeric pick values separately from authoritative validity flags. Reject/report malformed picked cells and define safe replacement semantics; never repair the three unpicked cells from CSV truth.

Regression test: NaN/Inf under picked flags, stale finite values under unpicked flags, and Merge onto an existing finite cell. Assert diagnostics/counts and safe old-cell behavior.

Should existing tests have caught it?: The current H02503 regression correctly verifies the three mask exclusions and ordinary values but does not test malformed numbers under valid picked codes.

Evidence: native-edge-evidence.json: picked_nan

Confidence: High

Low

L01 — Save closes the Symbol Editor without warning about unapplied form edits

Severity: Low

Module / files and lines: geoworkbench/symbol_catalog_dialog.py:140–143,467–479

Function / class: SymbolCatalogDialog.save

Exact problem: Fields update the live preview, but Save serializes only the working catalog and does not apply/check dirty form fields. The separate Apply to the symbol button is required.

Why it matters: Users can see a changed preview, click Save and lose that form edit without warning. The explicit Apply button makes this a contained usability issue rather than catalog corruption.

Reproducible scenario: Change GAS description in the form to Edited in form and click Save without Apply. save returns true and closes; reloaded description is still Gas Well.

Recommended correction: Apply/validate a dirty current form on Save, or explicitly warn that the form has unapplied changes and keep the dialog open. Preserve Cancel semantics.

Regression test: Edit→Save, edit→Apply→Save, invalid edit→Save, changing selection with dirty fields and Cancel; make retained/dropped edits explicit.

Should existing tests have caught it?: Current editor persistence tests call apply_form before save, so they do not exercise the common direct-Save path.

Evidence: editor-evidence.json: save_without_apply

Confidence: High

What the native Kingdom validation actually establishes

The application menus call kingdom.read_native, which calls kingdom_native.read_horizon or read_fault. The older CSV path remains a separate validation/development utility; no inspected native menu substitutes CSV output for binary parsing. The new CFB reader reads the FLT container and streams directly. HZN values and two-bit validity masks are decoded from native tiles, including a tile split by an index block.

H02503: the automated test compares native coordinates/values against exported truth and explicitly requires the absent cells to be **(242,309), (243,309), (244,309)**. The native mask wins; these three stale finite-valued cells are not repaired from CSV. The inspected parser never consults CSV truth in production. It returns **1,830 native picks**. This passes; M05 is a different case involving a corrupted picked value.

Names: native labels are authoritative. The tests explicitly expect **DTS1**, **Rob L-1 LS dtse1**, and **Rob L1** where exported/project-database names differ. These differences are not defects and no renaming “correction” is recommended.

Orientation/X/Y: the H02503 test numbers internal grid indices using the stated survey origin and compares bin placement/X/Y to the exported coordinates. This supports the tested unit-increment survey. The import UI requires a unit, rejects time/depth domain mismatch and keeps the selected origin/unit/factor in provenance. It cannot validate

an arbitrary origin from a file that contains no X/Y or line numbering. C04/M02 identify missing protection at that boundary.

Fault topology on first import: the tests compare every tested stick ID, header word, sequence position, line/trace coordinate, X/Y and value to CSV rows. The shipped fixtures contain four nonempty faults: F03301 (1 stick/2 points), F01804 (16/82), F01000 (316/1,842) and F01100 (23/271), plus one empty fault and one fault-polygon file. First-import topology is meaningful and the extra fields survive persistence. C01 is the subsequently merged-identity case.

The reported all-26 comparison is not reproduced by this archive. docs/NATIVE_IMPORTS.md claims all 26 match, but tools/make_native_fixtures.py:NATIVE_FAULTS retains only six files, including the empty/polygon cases. The full 26 native files and complete independently decoded truth were not supplied. The current tests validate their subset, not the entire claim. Do not treat the unsupported breadth of that claim as proof that any omitted fault is wrong.

Independence of expected values

tools/make_native_fixtures.py:159–232 copies FLT bytes and selects rows from external flt/fault_picks_all.csv. For horizons it reads separate hzn/horizon_*.csv truth tables; the expected scientific values are not generated by calling the native parser. Therefore the checked-in generator does **not** show the circular value test parser → expected values → parser comparison.

There is, however, coupling in fixture preparation: it imports native parser helpers to locate/cut HZN tiles, modifies H01386 to keep selected pages and samples every tenth retained exported pick. This can share coverage blind spots with the decoder. Preserve the external truth, add at least one untouched binary sample and held-out fixtures across directory/index boundaries, and retain the original external decoder/export provenance and hashes. The original external decoding process was not available here, so its independence cannot be forensically certified from the selected CSV files alone.

Identity, symbol and persistence checks that passed

Check	Evidence and result	Limits
IHS 298 completion identity	External probe imported 4 2401123450000/0001/0002 with the same month and different volumes. Three wells and three distinct production rows retained their IDs/values after reimport and close/reopen.	Applies to these explicit full-API cases; not certification of every malformed 298 layout.
Reimport conflict behavior	Reimport found three identical rows and created no wells; editing one completion produced one conflict and two identical rows.	A fresh plan is used for the second import.
No 10-digit collapse in 298	ihs298.plan builds an exact API14 dictionary and uses api14; no WellMatcher or [:10] shortcut appears in this identity path. api14 pads 10/12 digits in general helpers, but native 298 additionally requires an original full 14-digit value.	production.WellMatcher retains a short-API fallback for other workflows; its existence is not evidence that 298 uses it.
New 298 well metadata	_new_well stores full API, lease-derived name, available operator/field and source/entity/lease notes; default symbols are applied. Tests verify available metadata and provenance.	No claim that unavailable fields are invented or that well DB primary keys literally equal API strings; internal integer IDs are expected.

Check	Evidence and result	Limits
298 location datum	Location is added only for a newly created well with a selected datum and supplied coordinates, transformed to WGS84/project coordinates. Existing native tests cover creation with/without a datum.	Actual Windows/CRS-grid distribution was not exercised.
298 rollback	Cancellation after new-well creation rolled back both wells and monthly rows to zero in the external probe. Existing tests also cover cancellation.	Cancellation responsiveness during the creation loop still needs H04.
297 mapping path	symbol_for calls the centralized resolver. Existing tests verify mapped status assignment, unmapped existing-well preservation, unmapped new-well default and diagnostics.	H02 can corrupt that central mapping despite this correct call path.
Exactly 29 shipped active codes	External catalog inspection matched the requested ordered list exactly; retired DRYSO/AOIL/SIOIL/STAR/OTHER/PR do not reappear through the inspected fallback.	A user may intentionally add/import custom codes; that is separate from shipped builtins.
Shared geometry	Actual primitive tuples were equal for PLUGOIL=PAOW, DRILWELL=DERRICK and DRL=DERRICK; the existing tests also compare distinct descriptions/codes.	PLUGINJ remains awaiting user visual confirmation; it is not classified as an incorrect symbol.
Legacy symbols	Current tests open an old-code well, show a neutral legacy representation, preserve its code/ID on save and explicitly reassign it. External migration probe preserved DRYSO/58 on repeated reopen.	Preserving a legacy value does not reactivate it in the picker.
User settings ownership	config_path uses APPDATA/LOCALAPPDATA or a user-home fallback; valid edits are staged via a sibling temporary JSON file then replaced. The editor uses a deep working copy and Cancel discards it.	Startup validation H03, rejected edits H02 and unapplied form L01 remain exceptions.
Packaged WSD resolution	well_symbols.resource_path honors _MEIPASS; _find_wsd supports the spaced WSD name and suffix fallback. Build_Windows_EXE.bat includes the WSD and icon/data resources for all applications.	Configuration inspection and a source self-test are not a frozen-Windows acceptance run.

Check	Evidence and result	Limits
Provenance migration	External probe removed both seismic provenance columns from an otherwise schema-29 DB, then reopened it twice. Both columns returned and legacy well IDs/API/symbol values were unchanged.	This is a controlled migration-shape check, not an authentic v0.9.3 client-project upgrade.
Native save/reopen	Supplied tests compare the HZN grid, 16 FLT stick groups/sequences, native IDs, source filename and selected origin after save/load. External merged-fault probe performs an actual connection close/reopen.	The merge topology/provenance defect persists across that reopen; successful new-import tests do not cover it.

The current schema migration inspects columns before ALTER TABLE and uses IF NOT EXISTS for indexes, including idx_formation_tops_source with NOCASE. The previous top-source lookup optimization remains in database.py:2887–2902: a bounded SELECT ... WHERE source=? COLLATE NOCASE LIMIT 1 rather than rebuilding the full source list for each pick. No destructive schema change was established in the targeted review. Because the v0.9.3 source tree was not provided, a complete version-to-version DDL diff cannot be asserted.

IHS 297 coverage and remaining risks

The parser reads physical lines sequentially and reports unsupported/unknown record families; the reduced 57-well fixture includes all record codes selected from its source export. A/C/DA headers and location, ET tops, U2 surveys, I/ID/IN cores, F/G IP tests, H FM tests, treatments and recoveries have explicit destinations/tests. The test suite verifies that tested intervals do **not** create automatic perforations. No new fallback was found that silently guesses an active symbol for an unmapped IHS status.

Both 297 and 298 use an outer db.atomic() transaction. Nested helper transactions become savepoints; helper execute calls do not commit the whole import early. Existing cancellation tests pass and the independent 298 rollback probe confirms all-or-nothing behavior. That is useful protection against an exception halfway through the write. It does not protect against C03, where a deletion is an intentionally executed and successfully committed operation. Some individual invalid-child errors are logged/continued after the source's previous children were cleared, so replacement validation must precede deletion.

“Streaming” needs precision: both parsers read files line-by-line, but accumulate Parsed wells/entities, children/months and then a plan for preview. They are not constant-memory end-to-end pipelines. Native HZN/CFB reads retain complete byte buffers, and native faults build per-node dictionaries in the adapter. No production-size original IHS/Kingdom files or long-session client workloads were provided for a peak-memory certification. These are bounded assessment limits, not an unsupported claim that normal imports leak.

The measured performance defect is H04: a repeated scan of all wells before every newly inserted API, despite an indexed final lookup. The source lookup optimization should be preserved while addressing this separate cost. For 298 the large existing-well plan builds a single API14 dictionary and caches existing monthly rows per matched well; those are intentional bulk operations rather than an accidental 10-digit lookup. The probe observed one commit, so excess per-month commits are not the diagnosed problem.

Windows and error-handling qualification

The current build includes data, spaced WSD resources, icons and PDF/SVG backends and collects native PDFium dependencies. User catalog writes target the user settings folder rather than the installation directory. Native binary reads use bytes/little-endian layouts and suffix selection is case-insensitive. Native dialog filters include both upper/lower HZN and FLT patterns. Those are supported source observations.

H01 is an actual source-command failure whose same assertion is invoked by the Windows build gate. It does not establish that the executable was built/run on Windows or that DLL resolution failed. After fixing it, run the actual frozen applications on a clean Windows account with no Python/Poppler and an unwritable installation directory; open the spaced WSD resource, edit/restart the catalog, import native files from paths with spaces, save/reopen and exercise drag/drop and PDF/SVG export.

Malformed-file handling is incomplete (M01/M05), while signature rejection, empty-fault/polygon diagnostics and ordinary invalid-unit/domain selection do have tests. Malformed user-settings types are a distinct startup problem (H03). WSD tests exercise valid CRLF/LF parsing and round-trip/import; malformed blocks, invalid numeric scale strings and partial-import recovery deserve dedicated diagnostic tests. Do not infer comprehensive robustness from the valid built-in WSD round trip.

Test quality and interpretation

The suite contains meaningful tests: independent exported value comparisons, real DB persistence, GUI editor operations, IHS child handling, rollback, legacy symbols and numerical/render equality. Its gaps are specific:

- Single-import topology and reopen tests do not exercise same-ID sticks through Merge.

- The editor duplicate-code test checks the entry, but not mappings/geometry references modified before rejection.

- Same-complete-file idempotence checks compare counts, not child identifiers, values, physical units or partial-delivery preservation.

- Native app-flow stand-ins count saves, so they cannot expose real save-error propagation or success-status overwriting.

- Fresh-marker geometry tests do not validate existing Map artist invalidation after catalog edits.

- test_native_ihs298.py uses assertTrue(months_base & months_other or months_base), which succeeds whenever months_base is nonempty; it does not by itself prove that the two completions share a month. The independent probe here explicitly gives all three the same month.

- Signature rejection is not coverage of inner directory/sector truncation, NaN picks or unsupported numbering increments.

- Documentation says all 26 faults; bundled tests check a smaller selected set. Test scope and externally reported validation scope should be distinguished.

- Config-text checks of build commands do not execute the real self-test. H01 survives despite the good 29-code contract test.

- The exact Windows/Tk/real-raster baseline and earlier process crash still need reproduction in the original release environment; a font substitution here is not evidence of that crash.

No tests were weakened, deleted or changed during the audit. No style-only code cleanup is proposed. No recommendation to merge renderers, change geological equations, silently infer units or rework the UI architecture follows from these findings.

Tests to add

1. **Data-loss gates:** header-only 297 update preserves prior children; empty/outside horizon Replace preserves the prior grid and DB; metre-target 297 records match hand conversions.

2. **Fault identity:** New/Merge/Replace with repeated native stick IDs, distinct source files and identical reimports; verify ordered child groups and complete provenance after an actual close/reopen.

3. **Real release entry point:** execute source and frozen Geo --self-test with exactly 29 builtins and valid user changes; keep missing-resource detection meaningful.

4. **Editor transactionality:** compare the entire working catalog before/after rejected renames; save/restart and resolve IHS statuses. Cover dirty-form Save/Cancel and duplicate SYM policy.

5. **Durable-save errors:** inject failure in native import persistence and assert the durable state, visible unsaved warning, retry and return outcome.

6. **Catalog startup:** malformed typed JSON, bad render definitions/mappings and future settings versions through real Geo/Seismic startup with safe recovery.

7. **Native corruption/placement:** truncate at internal boundaries, corrupt FLT chains/counts, non-finite picked values, unsupported numbering intervals and unit/domain mismatch. Preserve H02503's exact three exclusions.

8. **Validation breadth:** untouched HZN sample plus held-out 26-fault/native-export pairs with source hashes and original decoder/export provenance; expected values must not be derived from the parser under test.

9. **Live views and scale:** save catalog styles with already-open Map/sections; assert artist updates. Trace 100k-well imports to bound full-table refreshes, memory and cancellation latency.

10. **Original Windows baseline:** run the exact branch commit and real-raster fixtures, repeat the earlier crashing raster sequence without weakening assertions, then qualify the frozen installer on a clean machine.

Recommended correction order

First protect the destructive boundaries (C01–C04) and preserve regressions as synthetic tests. Then repair the stale build gate (H01), atomic catalog validation/rejection (H02/H03) and persistence-result propagation (H05). Address bulk API refresh/cancellation (H04) without relaxing exact identity checks. Complete native malformed-data/placement protection and live-style invalidation (M01–M05). L01 can be a contained follow-up if the required Apply step is made explicit in the meantime.

Retain the approved 29-code catalog, centralized mappings, correctly isolated full API14 production identity, native mask authority, independently sourced CSV expectations and standard transaction services. PLUGINJ should await user visual confirmation. No evidence justifies a broad rewrite.

Overall assessment

Do not release. Four reproduced workflows can lose or misassign geological/project data, and the configured Geo build smoke test fails against the intentional 29-symbol catalog. Fix these narrowly, add the missing boundary tests and requalify the actual Windows build. The passing API14, native-value and first-import topology checks provide a useful baseline; they do not neutralize the destructive cases found here.